AgroTrace — AI/ML + Real Blockchain Production Build Plan (v2, corrected to actual stack)

Purpose: Build-ready spec for an agentic coding tool (e.g. Antigravity) to implement five systems on top of the **actual AgroTrace** codebase.

> **This version supersedes the earlier draft**, which incorrectly assumed NestJS + PostgreSQL/PostGIS. Corrected below against the real stack you provided. The single biggest change: **blockchain is currently 100% simulated** (a fake 40-char hex string generated on purchase) — Section 7 is rewritten to replace that with a real chain, wired into the transit/traceability flow that already exists end-to-end.

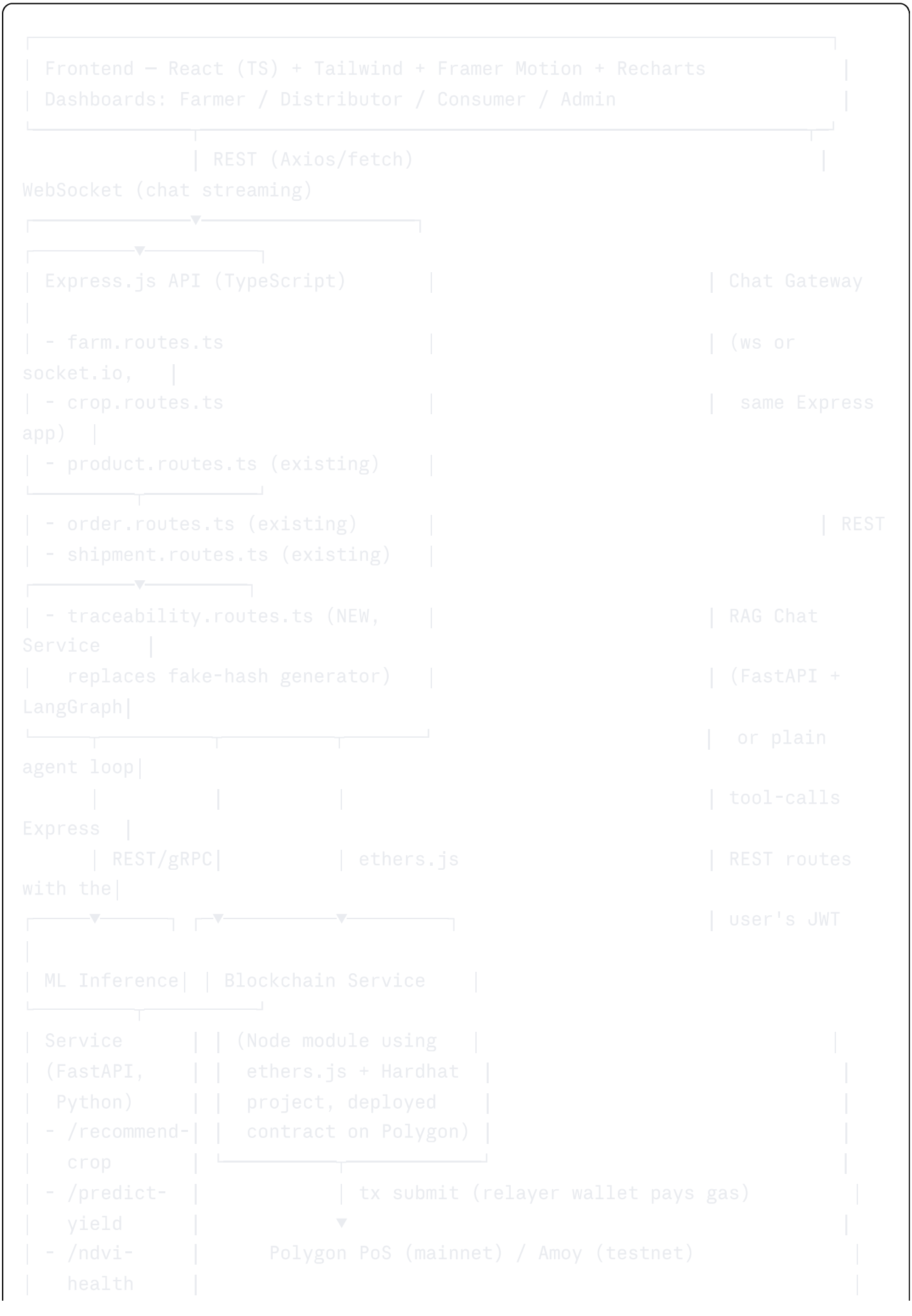
---

0. What Changed From v1 → v2, and Why

v1 assumed	Actually is	Impact
NestJS backend	**Node.js + Express.js (TypeScript)**	All controller/route examples rewritten as Express routers, not Nest modules/decorators.
PostgreSQL + PostGIS, Prisma (relational)	**MongoDB, via Prisma's Mongo connector**	Schema rewritten in Prisma-Mongo syntax (`@db.ObjectId`, no true FK joins, no native PostGIS polygons — geospatial handled via Mongo `2dsphere` index on a point, not a field polygon).
Only `Field`/`Crop`/`NDVIReading`/`YieldPrediction` existed	**Full marketplace + logistics already exists:** `Farm` (lat/lng), `Crop`, `Product` (SKU + QR), `Order`, `Shipment`, `SupplyChainEvent`, plus 4-role RBAC (Farmer, Distributor, Consumer, Admin)	New models plug into this existing graph instead of inventing a parallel one.
No weather/market integration assumed	**Open-Meteo (weather) and Agmarknet (Mandi prices) already integrated**	Crop Recommendation and Yield models reuse these live feeds instead of adding new weather APIs; ROI calc uses **real** Agmarknet prices,

v1 assumed	Actually is	Impact
		not a placeholder `MarketPrice` table.
Blockchain: to be built from scratch	**Blockchain traceability UI/UX flow already fully wired — but the "chain" itself is a `Math.random()`-style fake hash**	This is now a **replacement**, not a **greenfield build**: swap the fake-hash generator for a real on-chain call at each existing `SupplyChainEvent` write, with zero change to the frontend timeline UI.
Statistical yield/price model absent	**A linear regression already runs on 6 months of Agmarknet price history**, labeled "Statistical Yield Projections" but it is actually a **price trend model**, not a yield model	Flagged as a naming/scope issue below (Section 4.0) — keep it, rename it, and build the *actual* yield-prediction model separately.
NDVI already simulated fallback only	Confirmed: **NDVI fusion is currently visualization-only over synthetic data**	Section 5 unchanged in spirit, adjusted for Mongo storage and point-based farm location (no polygon) unless farmers are asked to draw one.

1. High-Level Architecture (corrected)



Why still keep ML in a separate Python service: Express/Node is not the right runtime for XGBoost/LightGBM/rasterio/GDAL/satellite processing. The Express backend calls the FastAPI service over internal REST exactly as it would call any other internal microservice — no framework mismatch, just an added service boundary.

---

2. Data Model Additions (Prisma + MongoDB — corrected)

Prisma's MongoDB connector does **not** support relational foreign keys/joins the way Postgres does — relationships are either **embedded documents** or **manual reference IDs** resolved with separate queries (or `$lookup` via raw aggregation). No PostGIS polygons; use a GeoJSON `Point` with a `2dsphere` index (raw Mongo command, since Prisma doesn't manage geospatial indexes — create it via a migration script using the native Mongo driver, not `prisma migrate`).

prisma

```
// Existing (inferred) — do not recreate, just referenced
// model Farm    { id String @id @default(auto()) @map("_id") @db.ObjectId, far
// model Crop    { id String @id @default(auto()) @map("_id") @db.ObjectId, far
// model Product { id String @id @default(auto()) @map("_id") @db.ObjectId, cro
// model Order   { id String @id @default(auto()) @map("_id") @db.ObjectId, pro
// model Shipment { id String @id @default(auto()) @map("_id") @db.ObjectId, or
// model SupplyChainEvent { id String @id @default(auto()) @map("_id") @db.Obje

// NEW models to add:

model NDVIReading {
  id            String   @id @default(auto()) @map("_id") @db.ObjectId
  farmId        String   @db.ObjectId
  captureDate   DateTime
  ndviScore     Float
  cloudCoverPct Float?
  source        String   @default("SIMULATED") // "OBSERVED" | "SIMULATED" | "I
  satelliteId   String?
  rawTileUrl    String?
  createdAt     DateTime @default(now())
}

model YieldPrediction {
  id               String   @id @default(auto()) @map("_id") @db.ObjectId
  cropId           String   @db.ObjectId
  predictedAt      DateTime @default(now())
  predictedYieldKg Float
  confidenceLowKg  Float?
  confidenceHighKg Float?
  modelVersion     String
  dataQuality      String   // "OBSERVED" | "PARTIAL" | "SIMULATED"
}

model CropRecommendation {
  id             String   @id @default(auto()) @map("_id") @db.ObjectId
  farmId         String   @db.ObjectId
  recommendedAt  DateTime @default(now())
  topCrops       Json     // [{crop, score, expectedYieldKg, expectedRoi}]
  inputsSnapshot Json
  modelVersion   String
}

model CropHealthAssessment {
```

```
  id           String   @id @default(auto()) @map("_id") @db.ObjectId
  farmId       String   @db.ObjectId
  assessedAt   DateTime @default(now())
  status       String   // "HEALTHY" | "STRESSED" | "CRITICAL"
  ndviTrend    Json
  cause        String?
  confidence   Float
  modelVersion String
}

model ChatSession {
  id        String   @id @default(auto()) @map("_id") @db.ObjectId
  userId    String   @db.ObjectId
  startedAt DateTime @default(now())
}

model ChatMessage {
  id        String   @id @default(auto()) @map("_id") @db.ObjectId
  sessionId String   @db.ObjectId
  role      String
  content   String
  toolCalls Json?
  createdAt DateTime @default(now())
}
```

Geospatial index (run once, outside `prisma migrate`, via the native driver):

javascript

```
db.Farm.createIndex({ location: "2dsphere" });
// Farm documents should carry: location: { type: "Point", coordinates: [lng, l
```

Extend existing `Crop` model (via a small migration script, since Mongo is schemaless underneath):

javascript

```
aiPredictedYieldKg: Number,        // NEW
aiPredictionConfidence: Number,    // NEW
aiModelVersion: String             // NEW
```

Extend existing `SupplyChainEvent`: keep `txHash` field (already exists) but change *what populates it* — see Section 7.

---

3. Model 1 — Crop Recommendation

Unchanged in modeling approach from v1, corrected on data sources:

- **Weather features**: pull from the **already-integrated Open-Meteo** call (reuse existing service/util, don't add a second weather client).
- **ROI feature**: use the **already-integrated Agmarknet** modal price for the farmer's state/commodity instead of inventing a `MarketPrice` table — call the existing Agmarknet integration (or the same cached data the price-trend feature already stores in MongoDB) rather than duplicating an external call.
- **Cold start**: public Kaggle-style crop recommendation dataset (N/P/K, temp, humidity, pH, rainfall → crop), fine-tuned continuously on AgroTrace's own outcomes (`Crop.expectedActualYieldKg` vs. what actually got listed as a `Product` and sold).
- **Serving**: `POST /recommend-crop` on the FastAPI ML service, called by a new `ml.service.ts` in the Express backend, result persisted to `CropRecommendation`.

Everything else (feature list, model choice = XGBoost multiclass → ranked list, training pipeline, MLflow registry, guardrails on low-confidence recommendations) is unchanged from v1 Section 3.

---

4. Model 2 — Crop Yield Prediction

4.0 Important naming clarification (new)

The existing **"Statistical Yield Projections"** feature (linear regression on 6 months of Agmarknet price history) is a **price trend model**, not a yield model — it forecasts whether Mandi prices are rising/falling/stable, which helps a farmer decide *when to sell*, not *how much they'll harvest*. Recommend:

1. **Rename it in the UI/code** to something like "Price Trend Forecast" to stop it being confused with the new yield model.
2. **Keep it as-is** — it's a legitimate, working feature and shouldn't be touched or replaced.
3. **Build the actual Yield Prediction model described below as a separate, new feature**, feeding the existing "Estimated vs Expected Yield" Recharts bar chart's missing third series: `aiPredictedYieldKg`.

4.1–4.6: Model design (mostly unchanged from v1, corrected for stack)

- Inputs: NDVI trend (from the new real ingestion pipeline, Section 5), Open-Meteo cumulative rainfall/GDD since planting, crop type/variety/planting date, soil params if collected, field's own past `Crop` history.
- Model: LightGBM regressor on tabular aggregates of the NDVI series (mean/slope/min/recovery-rate) + weather — same rationale as v1 (deep sequence models like LSTM/TFT are a documented Phase-2 upgrade once there's 2+ seasons of real data volume, not built now).

- Serving: `POST /predict-yield` on the ML service; Express route `GET /crops/:id/yield-prediction` calls it, persists to `YieldPrediction`, and returns `dataQuality` so the frontend Recharts line **visually distinguishes** `OBSERVED` (solid line) from `SIMULATED`/`PARTIAL` (dashed line + badge) predictions — this is the fix for the current chart implying more certainty than the underlying data supports.
- Retraining: weekly Airflow batch job; auto-promote only if RMSE beats the current production model on a fixed holdout.

---

5. Model 3 — NDVI-Based Crop Health Detection

5.1 Real ingestion (adjusted for point-based farm location)

Since `Farm` stores `lat/lng` (a point), not a drawn field polygon:

- **Option A (fastest to ship):** buffer the point by the farm's known/estimated area (`sqrt(areaHectares * 10000) / 2` as a rough square-buffer radius) to approximate a polygon for the satellite query. Good enough for a mean-NDVI-over-area estimate; label results as "approximate area" in the API response.
- **Option B (more accurate, slightly more product work):** add an optional "draw your field boundary" step in the Farm Registration UI, storing a proper GeoJSON polygon alongside the existing point. Recommended for Phase 2 once the point-based approximation is validated as workable.
- Ingestion job (Airflow, every 5 days matching Sentinel-2 revisit): query Google Earth Engine / Sentinel Hub for the buffered/drawn area, compute mean NDVI, write `NDVIReading(source: "OBSERVED", satelliteId: "Sentinel-2")`. Skip (don't fabricate) on high cloud cover; only fields with zero real history get an explicitly labeled `SIMULATED` placeholder.

5.2–5.4: Detection logic, serving contract, alerts — unchanged from v1

Rule-based first layer (NDVI thresholds/drop-detection) ships immediately with zero training data; ML-based cause attribution (water stress vs. pest vs. nutrient) is a Phase-2 layer once labeled outcomes exist. `POST /ndvi-health` on the ML service, Express route wraps it and writes `CropHealthAssessment`.

---

6. Model 4 — AgroTrace AI Chatbot (RAG, DB-aware)

6.1 Corrected integration point

Tool-calling agent (Claude via Anthropic API + LangGraph or a lightweight agent loop) whose tools map to **existing Express REST routes**, not NestJS controllers:

- `getFarmSummary(farmId)` → existing farm route
- `getNdviTrend(farmId, rangeDays)` → new NDVI route
- `getYieldPrediction(cropId)` → new yield route

- `getCropRecommendation(farmId)` → new recommendation route
- `getOrderStatus(orderId)`, `getShipmentTimeline(shipmentId)` → **existing** order/shipment routes — this is a bigger win than in v1's plan, since AgroTrace already has a full marketplace/logistics graph the bot can answer questions over ("Where is my order?", "Has my shipment been picked up?") for Consumer and Distributor roles too, not just Farmer.
- `getPriceTrend(commodity, state)` → wraps the **existing** Agmarknet-based price trend feature (Section 4.0) — the bot can now correctly answer "should I sell now?" using the real feature, correctly labeled.

6.2 Role-aware chatbot (new — reflects the 4-persona RBAC)

Because AgroTrace has 4 distinct roles, the chatbot's available tools and system prompt should be **scoped per role**:

- Farmer: farm/crop/yield/recommendation/price tools.
- Distributor: shipment/logistics tools only (no farm-financial data).
- Consumer: order/traceability/QR-verification tools only.
- Admin: broader read access across all of the above for dispute resolution, but still no write tools.

6.3 Security (unchanged principle)

Every tool call executes as the authenticated user (forward their JWT to the chat service → to Express), so existing RBAC middleware enforces access — the chatbot never gets a service-account bypass.

Everything else (pgvector or a lightweight vector store for the agronomy knowledge base, system prompt skeleton, streaming via WebSocket, audit logging to `ChatSession`/`ChatMessage`, rate limiting) is unchanged from v1 Section 6, with MongoDB substituted for the vector store note: **MongoDB Atlas Vector Search** is the natural choice here (no new infra dependency, since you're already on MongoDB/Atlas presumably), replacing the earlier "pgvector" suggestion.

---

7. Blockchain-Based Traceability — REPLACEMENT, not greenfield (major revision)

7.1 What exists today

- On every `Order` purchase, the backend immediately writes a `SOLD` `SupplyChainEvent` with a **`Math.random()`-style simulated 40-character hex string** stored in `txHash`, presented to the consumer as if it were a real blockchain transaction.
- Distributors already update `Shipment.status` through `PICKED_UP → IN_TRANSIT → DELIVERED`, each triggering a new `SupplyChainEvent` — this event-sourcing pattern is **exactly right** and doesn't need to change structurally.

- Consumers already scan a QR code (generated at `Product` creation) and see a chronological timeline built from `SupplyChainEvent` records — this UI/flow is correct and stays as-is.

The only thing wrong is that `txHash` is fake. This is a contained, well-scoped fix: swap the hash generator for a real on-chain write, keep every other layer (models, routes, UI, QR flow) untouched.

7.2 Chain choice

Polygon PoS (low gas, EVM-compatible, fast finality) — unchanged recommendation from v1. If a government/regulatory partner requires a permissioned ledger, Hyperledger Fabric is the fallback, but proceed with Polygon by default.

7.3 On-chain vs. off-chain split (unchanged principle, mapped to real models)

Data	Location
Full `SupplyChainEvent` document (who, what, where, notes)	MongoDB (off-chain), as today
SHA-256 hash of that event document	On-chain
Event type, batch/product SKU, actor ID, timestamp	On-chain (small, cheap, public timeline)
Product photos / quality-inspection images	Existing storage (S3/Cloudinary/whatever is already used for QR/product images); hash also anchored on-chain

7.4 Smart contract (unchanged from v1 — still valid against this stack)

Same `Traceability.sol` design as before: `recordEvent(batchId, eventType, metadataHash)`, `getBatchHistory(batchId)`, an `authorizedActors` allowlist populated by the backend after normal RBAC login (no separate on-chain KYC step), and a **relayer pattern** so farmers/distributors/consumers never touch a crypto wallet or pay gas — the Express backend holds a funded service wallet and signs on their behalf via `ethers.js`.

7.5 Corrected backend integration (Express, not Nest)

```
Existing flow (today):
  Order/Shipment status changes → SupplyChainEvent.create({ txHash:
fakeHash() })

New flow:
  Order/Shipment status changes
    → SupplyChainEvent.create({ txHash: null, chainStatus: "PENDING" })
    → blockchainService.recordEvent(batchId, eventType,
sha256(eventMetadata))
        (ethers.js call to deployed contract via relayer wallet)
    → on receipt: SupplyChainEvent.update({ txHash: realTxHash, chainStatus:
"CONFIRMED" })
    → background job periodically re-verifies: recompute hash from the stored
      MongoDB document, compare to on-chain metadataHash, flag mismatch as a
      tamper alarm if they diverge
```

This is a **drop-in replacement inside whatever function currently calls the fake-hash generator** — same call site, same event model, same consumer-facing timeline UI. The QR verification page's "link to blockchain" now points to a real PolygonScan transaction instead of a string that looks like one.

7.6 Testing & rollout

- Unit test the contract with Hardhat (access control, event emission, batch ordering).
- Deploy to **Polygon Amoy testnet** first; run the full existing purchase → shipment → delivery flow against testnet to confirm the swap doesn't break the UI (it shouldn't — `txHash` is still just a string field to the frontend).
- Only move to mainnet after a short audit/review of the contract, given real transaction costs (still fractions of a cent on Polygon, but real money and real immutability once live).

---

8. MLOps & Infra (adjusted)

Concern	Tool
Experiment tracking / model registry	MLflow
Pipeline orchestration	Airflow or Prefect
Vector store for chatbot's agronomy KB	**MongoDB Atlas Vector Search** (not pgvector — no Postgres in this stack)
Model serving	FastAPI + Uvicorn, containerized, deployed alongside the existing Express app
Monitoring	Prometheus + Grafana; Evidently AI for feature drift
CI/CD	GitHub Actions: lint → test → smoke-train → Docker build → deploy
Secrets	Vault/cloud secrets manager for Open-Meteo/Agmarknet keys (already in use — just confirm rotation policy), Anthropic API key, relayer wallet private key

8.1 Repo structure

```
/agrotrace
  /apps
    /api                (existing Express.js backend)
    /web                (existing React frontend)
    /chat-service         (FastAPI + LangGraph RAG chatbot — NEW)
    /ml-inference         (FastAPI serving crop-rec, yield, ndvi-health —
NEW)
  /ml
    /pipelines            (training scripts, Airflow DAGs — NEW)
    /notebooks
  /blockchain
    /contracts            (Traceability.sol — NEW)
    /scripts              (deploy, verify)
    /test                 (Hardhat tests)
  /infra
    /docker
    /airflow-dags
  /docs
    this-plan.md
```

9. Phased Delivery Roadmap (adjusted)

Phase	Scope	Duration
Phase 0	Confirm real schema field names (`Farm`, `Crop`, `Product`, `Order`, `Shipment`, `SupplyChainEvent`), add new Mongo collections, set up MLflow/Airflow/Docker skeleton	1 week
Phase 1	NDVI real ingestion (point-buffer approximation first), rule-based crop health detection, label existing simulated fallback explicitly as `SIMULATED`	2–3 weeks
Phase 2	Rename existing price-trend feature; build actual Yield Prediction model (LightGBM), wire `dataQuality`-aware chart into existing Recharts bar/line charts	2–3 weeks
Phase 3	Crop Recommendation model, reusing existing Open-Meteo/Agmarknet integrations	2 weeks
Phase 4	**Blockchain swap**: deploy contract to Amoy testnet, replace fake-hash generator at the existing `SupplyChainEvent` write site, re-run full purchase→shipment→delivery flow against testnet, then mainnet	2–3 weeks (shorter than v1's estimate — this is a swap, not a greenfield build)
Phase 5	Role-aware RAG chatbot (Farmer/Distributor/Consumer/Admin scoped tools)	2–3 weeks
Phase 6	Hardening: drift monitoring, retraining automation, contract review before mainnet, load testing	2 weeks

Total: roughly **12–16 weeks** (slightly shorter than v1's 14–18 weeks, since the marketplace/logistics/blockchain UI scaffolding already exists — Phase 4 is now a targeted replacement instead of a new build).

10. Security & Compliance Checklist (adjusted)

- [ ] Smart contract reviewed before mainnet — the immutability guarantee is only as good as the contract logic.
- [ ] ML inference and chat services are internal-only; Express remains the sole public gateway.
- [ ] Chatbot tools scoped per role (Farmer/Distributor/Consumer/Admin) **and** per authenticated user — no cross-farmer data leakage, no cross-tenant shipment visibility.

- [ ] `SIMULATED` NDVI/yield data never reaches a lending/insurance/subsidy decision without explicit flagging.
- [ ] Relayer wallet private key in a secrets manager, balance-monitored, gas-price-capped.
- [ ] No PII or farm GPS coordinates written on-chain — only hashes and pseudonymous actor IDs.
- [ ] Verify the current fake-hash `SOLD` events already in the database are either backfilled with real anchoring or clearly migrated/relabeled as legacy/demo data before go-live, so historical QR scans don't show a broken or misleading trail.

---

11. What to Hand to Antigravity, Concretely (adjusted)

1. This document.
2. Read access to the actual Prisma schema file and the Express route files for `farm`, `crop`, `product`, `order`, `shipment`, and wherever the current fake `txHash` generator lives (search for something like `crypto.randomBytes` or a hex-string template near the `SOLD`/shipment-status handlers) — that's the exact line Phase 4 replaces.
3. Confirm whether MongoDB is hosted on **Atlas** (needed to confirm Atlas Vector Search is available for the chatbot's knowledge base) or self-hosted (in which case substitute a standalone vector DB like Qdrant/Weaviate).
4. Confirm the Agmarknet and Open-Meteo integration code paths so the Crop Recommendation and Yield models call the *existing* clients instead of duplicating them.
5. Build **Phase 0 + Phase 1 first**, pause for review — same reasoning as v1: NDVI real ingestion is the foundational dependency for both the Yield and Health models, and the point-vs-polygon farm location decision (Section 5.1) should be confirmed with you before it's baked into the ingestion pipeline.
6. Build **Phase 4 (blockchain swap) second**, independent of the ML phases — it's the most contained, highest-integrity-impact fix (removing fake data presented as real) and doesn't depend on any ML model being ready.